# 10. 비전 트랜스포머

**비전 트랜스포머(Vision Transformer) : 트랜스포머 구조를 이미지 처리에 적용한 모델**

- 자연어 처리에서 사용하던 트랜스포머 구조를 이미지 분류에 적용
- CNN 처럼 합성곱 연산으로 지역 특징을 추출하는 대신, 이미지를 패치 단위로 나누고 패치 사이의 관계를 어텐션으로 학습

## # ViT

**ViT(Vision Transformer)**

- 자연어 처리 분야에서 트랜스포머 모델이 큰 성능 향상을 이끌어낸 아이디어를 이미지 분야에 적용
- 이미지를 작은 패치로 분할한 뒤 각 패치를 하나의 토큰처럼 사용
- 패치 임베딩과 위치 임베딩을 더한 값을 트랜스포머 인코더에 입력
- 분류를 위해 [CLS] 토큰을 사용
- BERT 가 문장을 토큰 단위로 입력하는 것처럼 ViT 는 이미지를 패치 단위로 입력

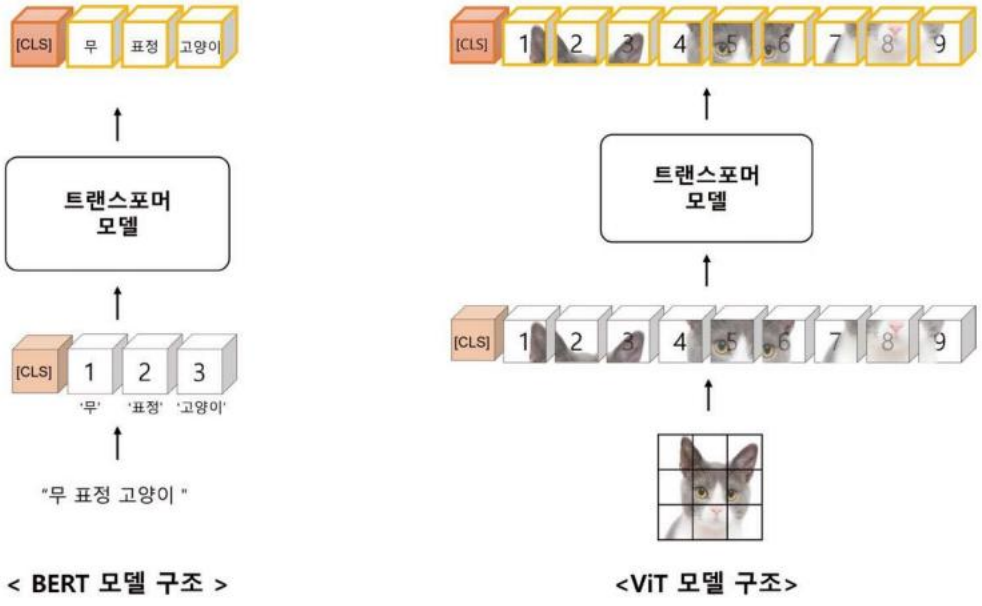



그림 10.2 BERT의 ViT 모델 구조 차이

## # 합성곱 모델과 ViT 모델 비교

**합성곱 신경망**

- 필터를 이용하여 이미지의 지역적인 특징을 추출
- 초기 계층에서는 저수준 특징을 추출하고, 계층이 깊어질수록 고수준 특징을 추출
- 각 필터가 확인하는 영역이 제한되어 있어 멀리 떨어진 영역의 관계는 여러 계층을 거친 뒤 반영

**ViT**

- 이미지를 여러 패치로 분할한 뒤 각 패치를 토큰처럼 처리
- 셀프 어텐션을 통해 이미지 내 패치 간 관계를 학습
- 멀리 떨어진 이미지 패치 사이의 관계를 직접 반영할 수 있음

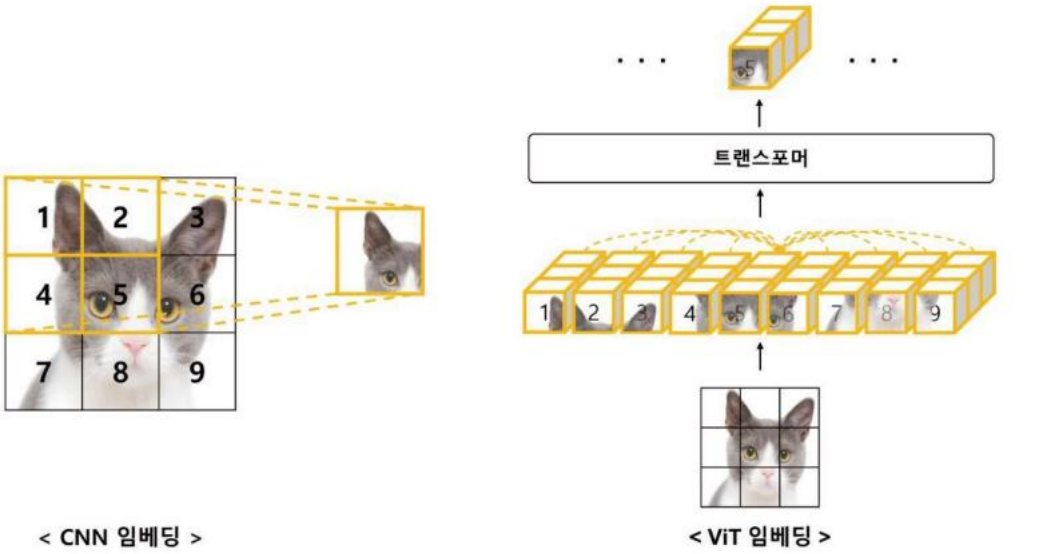



그림 10.3 합성곱 신경망과 ViT 계층의 차이

**수용 영역(Receptive Field, RF)**

- 하나의 출력이 계산될 때 영향을 미치는 입력 이미지 영역
- 합성곱 신경망은 계층이 깊어질수록 수용 영역이 커짐

**어텐션 거리(Attention Distance)**

- 셀프 어텐션에서 하나의 패치가 어느 정도 떨어진 패치까지 관계를 확인하는지를 나타냄
- ViT 는 이미지 전체 패치 사이의 관계를 어텐션으로 계산할 수 있음

## # ViT 의 귀납적 편향

**귀납적 편향(Inductive Bias)** : 학습 시 모델이 가지고 있는 가정(Assumption)

- 합성곱 신경망은 지역성(locality), 이동에 대한 불변성 등 이미지에 적합한 귀납적 편향을 가짐
- CNN 은 인접 픽셀 사이의 관계를 강하게 반영하고, 위치가 이동해도 같은 특징을 인식하는 구조를 가짐
- ViT 는 합성곱 연산을 사용하지 않으므로 CNN 에 비해 이미지에 대한 귀납적 편향이 적음
- 패치 사이의 관계를 데이터로부터 직접 학습해야 하므로 충분한 학습 데이터가 중요

## # ViT 모델

- 입력 이미지를 일정한 크기의 패치로 분할

- 각 패치를 일정한 길이의 벡터로 변환하여 패치 임베딩 생성
- [CLS] 토큰과 위치 임베딩을 추가
- 트랜스포머 인코더를 반복 통과
- 최종 [CLS] 토큰의 표현을 이용해 이미지 분류

→ 이미지 → 패치 분할 → 패치 임베딩 + 위치 임베딩 → 인코더 → 분류

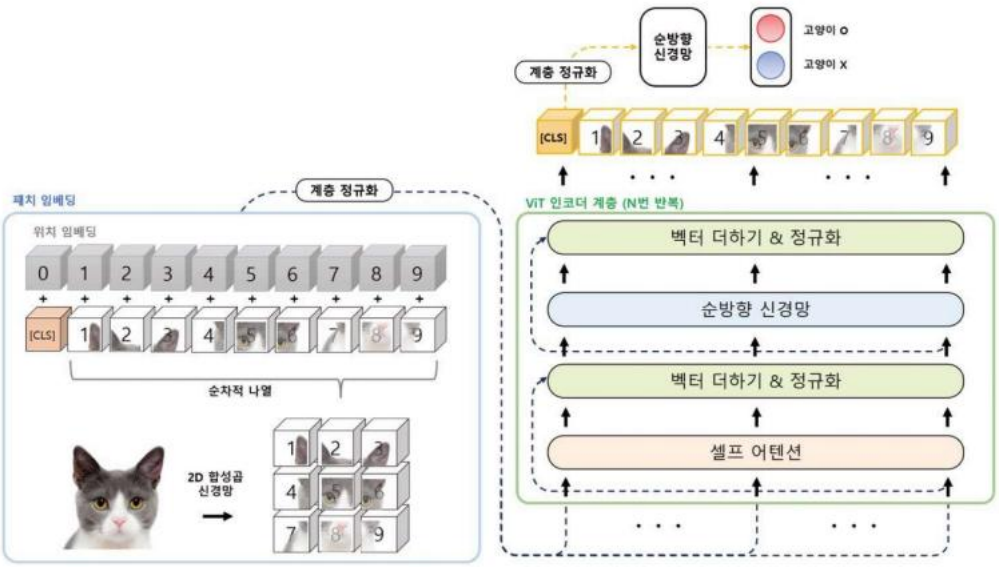



그림 10.4 ViT 모델 구조

## # 패치 임베딩

### 패치 임베딩(Patch Embedding)

- 입력 이미지를 작은 패치로 분할하고 각 패치를 일정한 길이의 벡터로 변환
- 패치 크기는 커널 크기(kernel size)와 스트라이드(stride)를 이용해 결정
- 교재 예시 : 9×9 이미지에 kernel size=3, stride=3 → 한 축에 3 개, 전체 3×3=9 개의 패치

**- 패치 개수 계산 : (image size - kernel size) / stride + 1**

### Classification Token, [CLS]

- 패치 임베딩 앞에 추가되는 특별한 토큰
- 이미지 전체의 정보를 대표하는 역할을 하며 최종 분류에 사용

### 위치 임베딩(Position Embedding)

- 패치의 순서와 위치 정보를 모델에 전달
- 패치 임베딩에 위치 임베딩을 더해 인코더에 입력

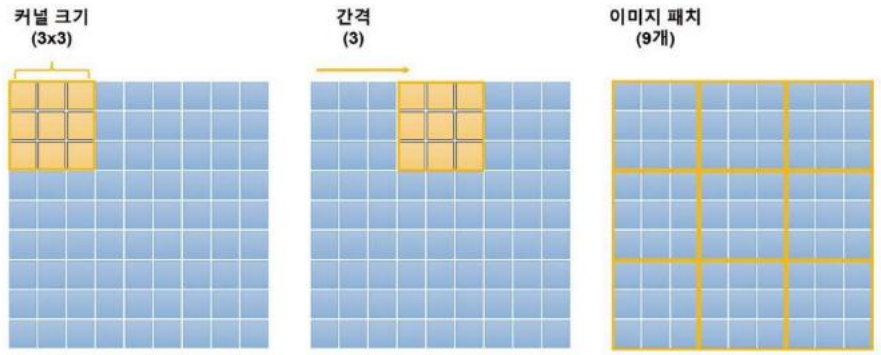


그림 10.5 이미지 패치 생성 과정

## # 인코더 계층

- 패치 임베딩을 입력으로 받아 패치 사이의 관계와 이미지 특징을 학습
- 멀티헤드 셀프 어텐션과 MLP 를 반복하여 특징 표현을 갱신
- 레이어 정규화와 잔차 연결을 함께 사용
- ViT 는 동일한 인코더 계층을 여러 번 반복
- 자연어 처리처럼 순차적으로 출력하는 것이 아니라 이미지 전체 특징을 한 번에 처리하여 분류

## # 모델 실습 - 데이터 준비

**subset_sampler()**

- FashionMNIST 전체 데이터 중 각 클래스에서 사용할 샘플 수를 제한
- 훈련 데이터 : 클래스별 1,000 개 → 총 10,000 개
- 테스트 데이터 : 클래스별 100 개 → 총 1,000 개
- defaultdict(list)를 사용하여 클래스별 인덱스를 저장한 뒤 Subset 을 생성

**datasets.FashionMNIST**

- root : 데이터 저장 위치
- download=True : 데이터가 없으면 다운로드
- train=True/False : 훈련/테스트 데이터 선택

**classes / class_to_idx**

- classes : 클래스 이름 목록
- class_to_idx : 클래스 이름과 클래스 ID 의 매핑

## # 이미지 전처리

**AutoImageProcessor**

- 사전 학습된 모델이 사용한 이미지 크기, 평균, 표준편차 등의 전처리 정보를 가져옴

**transforms.Compose**

- 여러 전처리 과정을 순서대로 묶어 적용

**ToTensor**

- 이미지를 Tensor 로 변환

**Resize**

- 이미지를 모델의 입력 크기인 224×224 로 변경

**Lambda**

- FashionMNIST 는 1 채널 흑백 이미지이므로 동일 채널을 3 개 연결해 3 채널로 변경

**Normalize**

- 사전 학습 모델에서 사용한 image_mean, image_std 로 정규화
-

## # collator 와 DataLoader

**collator**

- DataLoader 에서 배치로 가져온 이미지와 레이블을 ViT 입력 형식으로 변환
- pixel_values : 전처리가 적용된 이미지 Tensor
- labels : 정답 클래스 Tensor

**DataLoader**

- 훈련 배치 크기 32, 검증 배치 크기 4 로 구성
- drop_last=True 를 사용하여 마지막 불완전 배치를 제거

## # 사전 학습된 ViT 모델

**ViTForImageClassification**

- 허깅 페이스에서 제공하는 이미지 분류용 ViT 모델 클래스
- 사전 학습 모델 : google/vit-base-patch16-224-in21k
- 기존 사전 학습 모델은 21,841 개의 분류 카테고리로 학습되어 있으므로 FashionMNIST 10 개 클래스에 맞게 분류기를 조정

**num_labels**

- 현재 데이터셋의 클래스 개수 지정 → 10

**id2label / label2id**

- 클래스 ID 와 레이블 이름 사이의 매핑 정의

**ignore_mismatched_sizes=True**

- 사전 학습된 분류기 크기와 현재 클래스 수가 달라도 모델을 불러올 수 있게 함
- 최종 classifier 출력 : Linear(in_features=768, out_features=10)

## # 패치 임베딩 구조 확인

- model.vit.embeddings.patch_embeddings 로 패치 임베딩 계층 확인
- patch_embeddings 는 Conv2d 를 이용해 16×16 패치를 생성
- 입력 : [32, 3, 224, 224]
- 패치 임베딩 : [32, 196, 768]
- [CLS] 토큰을 추가한 전체 임베딩 : [32, 197, 768]
- 224/16=14 이므로 14×14=196 개의 이미지 패치가 생성되고 [CLS] 토큰 1 개가 추가

## # TrainingArguments

**TrainingArguments** : 학습에 필요한 주요 하이퍼파라미터를 지정하는 클래스

- output_dir : 체크포인트 저장 위치
- save_strategy="epoch" : epoch 마다 체크포인트 저장
- evaluation_strategy="epoch" : epoch 마다 검증 수행
- learning_rate=1e-5 : 초기 학습률
- per_device_train_batch_size=16 : 장치별 훈련 배치 크기
- per_device_eval_batch_size=16 : 장치별 검증 배치 크기
- num_train_epochs=3 : 전체 학습 epoch
- weight_decay=0.001 : 가중치 감소
- load_best_model_at_end=True : 학습 종료 후 가장 좋은 체크포인트 로드
- metric_for_best_model="f1" : 최적 모델 선택 기준을 F1 로 지정
- logging_steps=125 : 로그 기록 간격
- remove_unused_columns=False : 입력 데이터 열을 임의로 제거하지 않음
- seed=7 : 난수 시드 고정

## # 성능 평가 지표

**evaluate.load("f1")**

- F1 점수를 계산하는 평가 모듈 로드

**np.argmax**

- 각 샘플에서 가장 큰 예측값의 클래스 인덱스를 선택

**Macro Average F1-Score**

- 각 클래스의 F1 점수를 동일한 비중으로 계산한 뒤 평균
- FashionMNIST 처럼 여러 클래스를 분류하는 문제의 성능 평가에 사용

## # Trainer 를 이용한 ViT 학습

**Trainer**

- 모델, TrainingArguments, 훈련/검증 데이터셋, data_collator, 평가 함수를 전달하여 학습 수행
- trainer.train()으로 학습 시작
- 모델 체크포인트, 성능 지표, 하이퍼파라미터 등이 output_dir 에 저장

## # ViT 모델 성능 평가

**trainer.predict()**

- 테스트 데이터셋에 대한 예측 결과 생성

**outputs.label_ids**

- 실제 정답 레이블

**outputs.predictions.argmax(1)**

- 예측값 중 가장 큰 값의 클래스 인덱스를 최종 예측 클래스로 사용

**confusion_matrix / ConfusionMatrixDisplay**

- 실제 클래스와 예측 클래스를 비교하여 혼동 행렬 생성 및 시각화

| Epoch | Training Loss | Validation Loss | F1 |
|---|---|---|---|
| 1 | 0.7082 | 0.6077 | 0.8905 |
| 2 | 0.4964 | 0.4781 | 0.8966 |
| 3 | 0.4078 | 0.4357 | 0.9231 |

- 교재 실험에서 약 92% 수준의 분류 성능을 확인

## # Swin Transformer

**스윈 트랜스포머(Swin Transformer)**

- 2021 년에 발표된 비전 트랜스포머 모델
- 기존 ViT 의 높은 계산량과 고정된 패치 구조의 한계를 보완
- 계층적 구조(Hierarchical)를 사용하여 저수준 특징에서 고수준 특징으로 점진적으로 학습
- 이동된 윈도우(Shifted Window)를 이용해 지역 단위로 어텐션을 수행

## # ViT 와 스윈 트랜스포머 차이

### ViT

- 이미지 전체 패치 사이의 셀프 어텐션을 계산
- [CLS] 토큰 사용
- 고정된 패치 크기를 유지
- 입력 이미지 크기가 커질수록 패치 수와 어텐션 계산량 증가

### 스윈 트랜스포머

- 이미지를 작은 윈도우로 나누고 각 윈도우 내부에서 셀프 어텐션 수행
- [CLS] 토큰을 사용하지 않음
- 패치 병합으로 계층적 구조 형성
- 다양한 이미지 크기를 처리하기에 유리

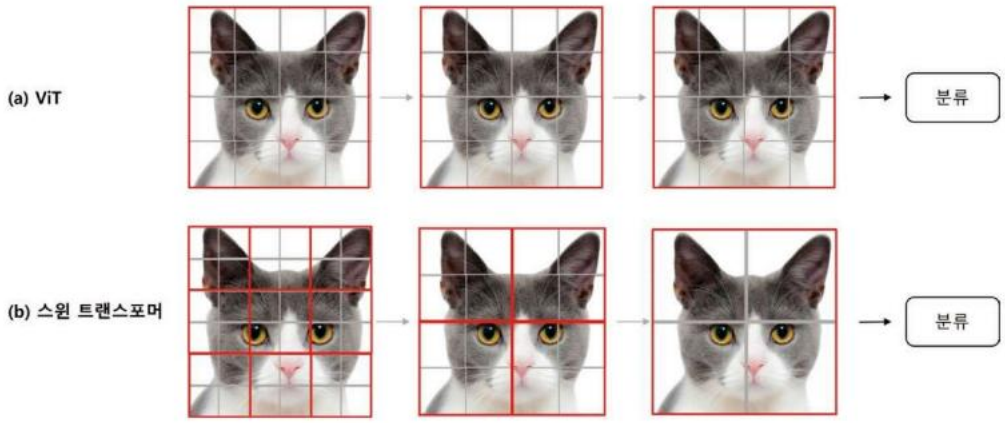



그림 10.6 ViT와 스윈 트랜스포머의 네트워크 구조

## # 스윈 트랜스포머 모델 구조

- 선형 임베딩(Linear Embedding) 이후 4 개의 스테이지로 구성
- 각 스테이지에는 여러 개의 스윈 트랜스포머 블록이 존재
- 스테이지 사이에서 패치 병합(Patch Merging)을 수행해 공간 크기는 줄이고 채널은 증가
- Stage 1 : H/4 × W/4 × C
- Stage 2 : H/8 × W/8 × 2C
- Stage 3 : H/16 × W/16 × 4C
- Stage 4 : H/32 × W/32 × 8C

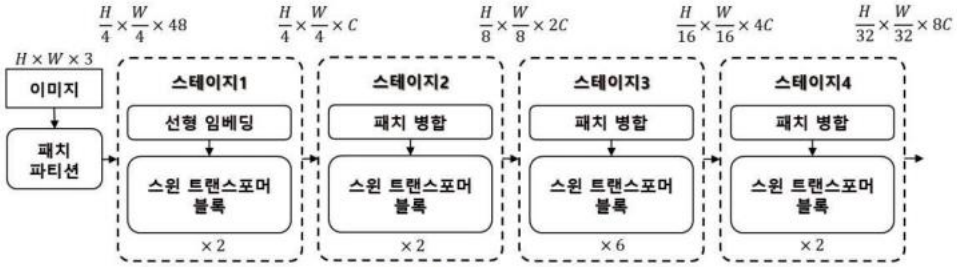



그림 10.7 스윈 트랜스포머 구조

## # 패치 파티션 / 패치 병합

### 패치 파티션(Patch Partition)

- 입력 이미지를 작은 사각형 패치로 분할
- 분할한 패치를 스윈 트랜스포머 계층의 입력으로 사용

### 패치 병합(Patch Merging)

- 인접한 2×2 패치들을 하나로 결합
- 공간 크기를 절반으로 줄이고 채널 수를 증가시켜 계층적 표현 생성
- 예 : [H, W, C] → 인접 패치 결합 후 [H/2, W/2, 4C] → 선형 계층을 거쳐 채널 조정

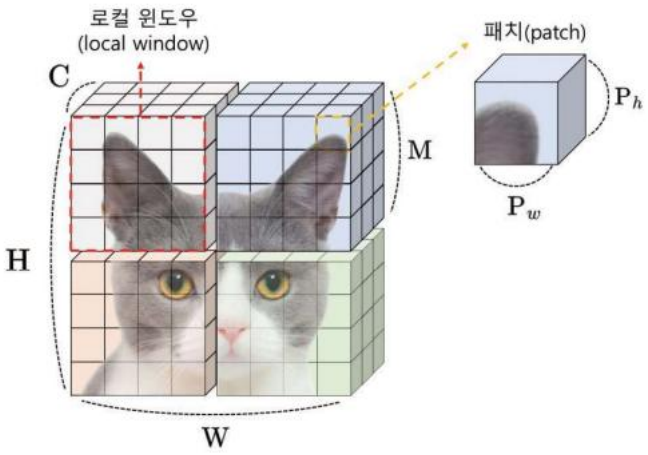



그림 10.8 패치 파티션 구조

## # 스윈 트랜스포머 블록

- W-MSA 와 SW-MSA 를 번갈아 사용
- 각 블록은 LayerNorm → Attention → 잔차 연결 → LayerNorm → MLP → 잔차 연결 구조

### W-MSA(Window Multi-head Self-Attention)

- 겹치지 않는 로컬 윈도우 내부에서만 셀프 어텐션 수행
- 전체 이미지에 어텐션을 수행하는 ViT 보다 계산량을 줄일 수 있음

### SW-MSA(Shifted Window Multi-head Self-Attention)

- 윈도우 위치를 이동시켜 이전 W-MSA 에서 서로 다른 윈도우에 있던 패치 사이의 관계를 학습
- W-MSA 와 SW-MSA 를 연결해 로컬 윈도우 간 정보 교환 가능

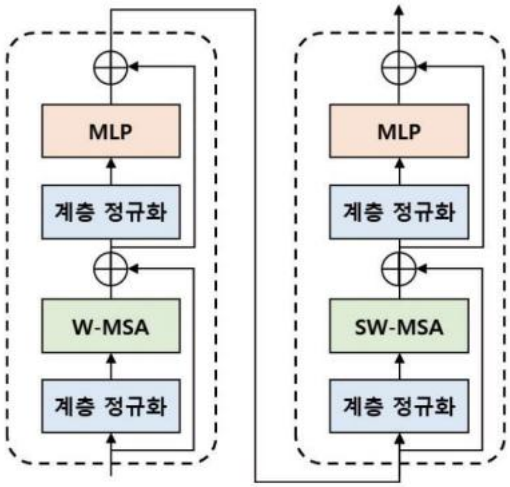



그림 10.9 스윈 트랜스포머 블록 구조

## # W-MSA 와 SW-MSA

### W-MSA

- 고정된 윈도우 안에서만 어텐션을 계산
- 윈도우별 연산을 효율적인 배치 계산(Efficient Batch Computation)으로 수행

### SW-MSA

- 윈도우를 일정 크기만큼 이동하여 새로운 윈도우 구성을 생성
- 교재에서는 순환 시프트(Cyclic Shift)를 이용

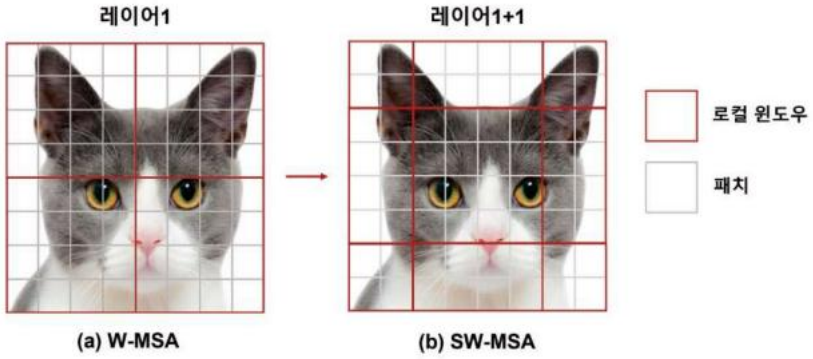



그림 10.10 W-MSA와 SW-MSA 어텐션 영역 비교

## # 순환 시프트 / 역 순환 시프트 / 어텐션 마스크

### 순환 시프트(Cyclic Shift)

- 윈도우를 이동시켜 서로 다른 윈도우에 있던 패치가 같은 윈도우에서 어텐션을 수행하도록 함
- 경계를 넘어간 영역은 반대편으로 이동

### 역 순환 시프트(Reverse Cyclic Shift)

- 어텐션 계산 후 이동시킨 패치 위치를 원래 위치로 복원

### 어텐션 마스크(Attention Mask)

- 순환 시프트 때문에 실제로 인접하지 않은 패치가 같은 윈도우로 들어오는 경우를 구분
- 서로 연결되면 안 되는 영역의 어텐션 계산을 제한

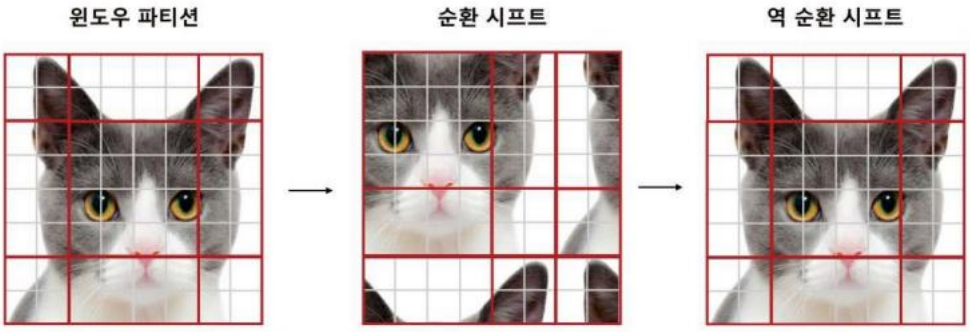



그림 10.11 순환 시프트를 활용한 SW-MSA 어텐션 계산 과정

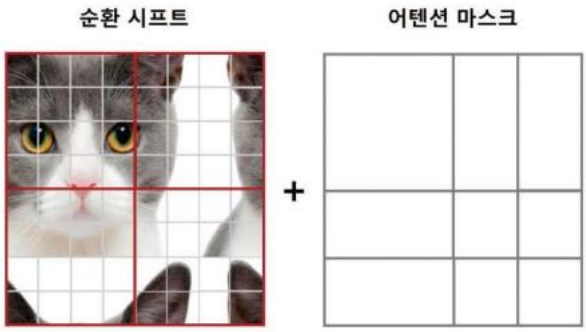



그림 10.12 순환 시프트 단계에서 마스크를 활용한 셀프 어텐션 구조

## # 상대적 위치 편향

### 상대적 위치 편향(Relative Position Bias)

- 윈도우 내부 패치 사이의 상대적인 위치 정보를 어텐션 계산에 추가
- 절대 위치가 아니라 패치 간 X 축, Y 축 거리 차이를 이용
- 상대적 위치 좌표를 하나의 index 로 변환해 relative_position_bias_table 에서 편향값을 선택

**Attention(Q,K,V) = SoftMax($QK^T$ / √d + B)V**

- B : 상대적 위치 편향
- 교재 예시는 window_size=2 로 4 개 패치 사이의 상대적 좌표와 index 를 계산

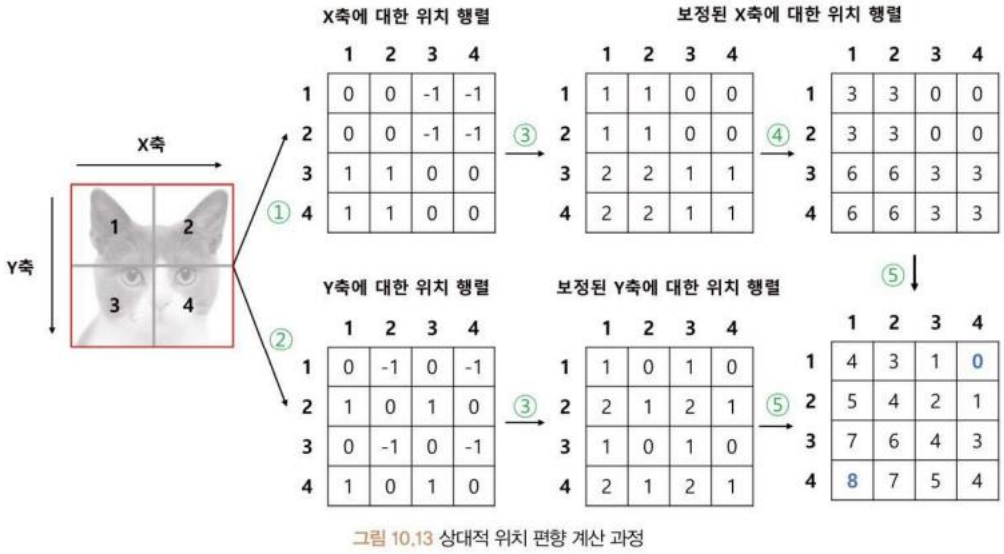



그림 10.13 상대적 위치 편향 계산 과정

## # 스윈 트랜스포머 실습 - 사전 학습 모델

- 교재 모델 : microsoft/swin-tiny-patch4-window7-224
- ImageNet-1k 를 이용해 224×224 이미지 분류에 사전 학습된 모델 사용

**SwinForImageClassification**

- 허깅 페이스의 스윈 트랜스포머 이미지 분류 모델 클래스
- FashionMNIST 10 개 클래스에 맞게 classifier 를 변경
- 모델 주요 구조 : swin → embeddings → encoder → layernorm → pooler → classifier

## # Swin 패치 임베딩

- patch_embeddings 의 projection 은 Conv2d(3, 96, kernel_size=4, stride=4)
- 입력 이미지 : [32, 3, 224, 224]
- 패치 임베딩 : [32, 3136, 96]
- 224/4=56 → 56×56=3,136 개의 패치 생성
- ViT 보다 패치 수는 많지만 초기 채널 수는 96 으로 작음
- 스윈 트랜스포머는 [CLS] 토큰을 사용하지 않음

## # SwinLayer

- 스윈 트랜스포머의 각 스테이지는 여러 개의 SwinLayer 로 구성
- SwinLayer 구성 : LayerNorm → Attention → Drop Path → LayerNorm → Intermediate → Output
- 첫 번째 블록은 W-MSA, 다음 블록은 SW-MSA 를 사용하며 반복

**Intermediate / MLP**

- 선형 계층과 GELU 활성화 함수로 구성

**GELU(Gaussian Error Linear Unit)**

- 스윈 트랜스포머의 MLP 에서 사용하는 활성화 함수

## # W-MSA / SW-MSA 출력 확인

- W-MSA 의 입력과 출력 차원은 동일하게 유지
- W-MSA 출력이 다음 SW-MSA 의 입력으로 전달
- 어텐션 방식은 다르지만 블록의 하이퍼파라미터와 출력 형태는 동일하게 구성

## # 패치 병합 실습

- 첫 번째 스테이지의 downsample 이 패치 병합 기능을 수행
- 패치 병합 전 : [32, 3136, 96] = 56×56 개의 패치
- 패치 병합 후 : [32, 784, 192] = 28×28 개의 패치
- 공간 패치 수는 1/4 로 감소하고 채널 수는 2 배 증가

## # 스윈 트랜스포머 모델 학습

- ViT 실습과 동일하게 FashionMNIST, AutoImageProcessor, TrainingArguments, Trainer 사용
- 달라지는 부분은 모델 클래스와 사전 학습 모델 이름
- 모델 : microsoft/swin-tiny-patch4-window7-224

| Epoch | Training Loss | Validation Loss | F1 |
|---|---|---|---|
| 1 | 0.3720 | 0.3046 | 0.8985 |
| 2 | 0.2579 | 0.2688 | 0.9138 |
| 3 | 0.2607 | 0.2416 | 0.9159 |

- F1 0.9159
- ViT 와 F1 점수는 비슷하지만 검증 손실값이 더 낮아짐

## # CvT

**CvT(Convolutional Vision Transformer)**

- 합성곱 신경망 구조에서 사용하는 계층적 구조와 트랜스포머를 결합한 모델

- ViT 의 셀프 어텐션에 합성곱 연산을 적용하여 이미지의 지역적 특징을 함께 활용
- 이미지 패치의 지역 정보와 트랜스포머의 장거리 관계 학습을 함께 사용

## # CvT 의 계층적 특징

- 스윈 트랜스포머와 마찬가지로 여러 스테이지의 계층적 구조를 사용
- 초기 스테이지에서는 저수준 특징, 뒤쪽 스테이지에서는 고수준 특징을 추출
- ViT 처럼 고정된 패치만 처리하는 대신 합성곱을 이용해 단계적으로 특징 맵의 크기와 채널을 변화

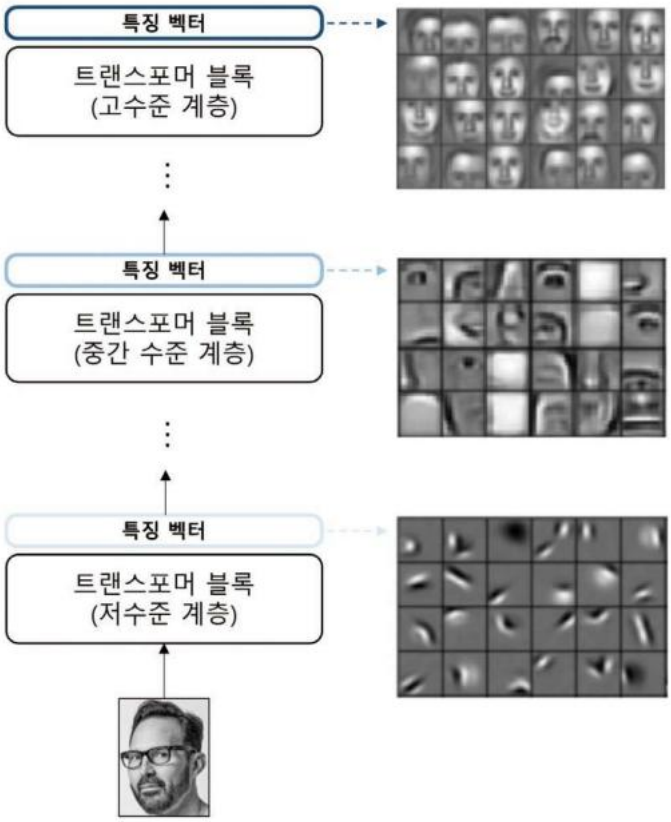



그림 10.14 CvT 모델의 계층적 특징 구조 예시

## # ViT 와 CvT 비교

| 구분 | ViT | CvT |
|---|---|---|
| 위치 임베딩 | 사용 | 사용하지 않음 |
| 토큰 생성 | 고정된 패치 임베딩 | 합성곱 토큰 임베딩 |
| 어텐션 | 선형 Projection 기반 Q/K/V | 합성곱 Projection 기반 Q/K/V |
| 계층적 구조 | X | O |

- CvT 는 합성곱 연산을 통해 이미지의 지역 정보를 반영하기 때문에 별도의 위치 임베딩 없이도 공간 정보를 활용

## # 합성곱 토큰 임베딩

### 합성곱 토큰 임베딩(Convolutional Token Embedding)

- 기존 ViT 의 단순 패치 분할 대신 2D 합성곱을 이용해 토큰 생성
- 이미지 패치를 일정 크기로 단순 분할하는 대신 커널과 스트라이드를 이용해 특징을 추출하며 다운샘플링
- 이미지의 지역적인 구조 정보를 토큰 생성 단계에서 반영
- 계층이 진행될수록 특징 맵의 공간 크기는 감소하고 채널 수는 증가

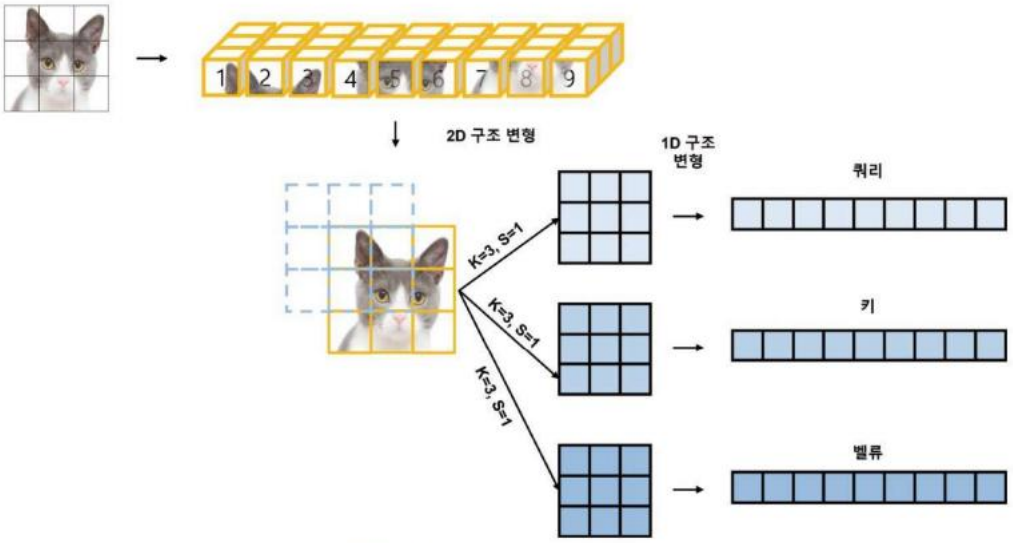



그림 10.15 2D 합성곱 기반의 어텐션 임베딩

## # 어텐션에 대한 합성곱 임베딩

### Convolutional Projection for Attention

- 셀프 어텐션에 필요한 Query, Key, Value 를 선형 계층이 아니라 2D 합성곱을 이용해 생성
- 합성곱을 적용한 뒤 텐서 형태를 어텐션 연산에 맞게 변환

### Squeezed Convolutional Projection

- Key 와 Value 생성 시 stride 를 크게 설정하여 토큰 수를 줄임
- 어텐션 계산량을 감소시키는 효과

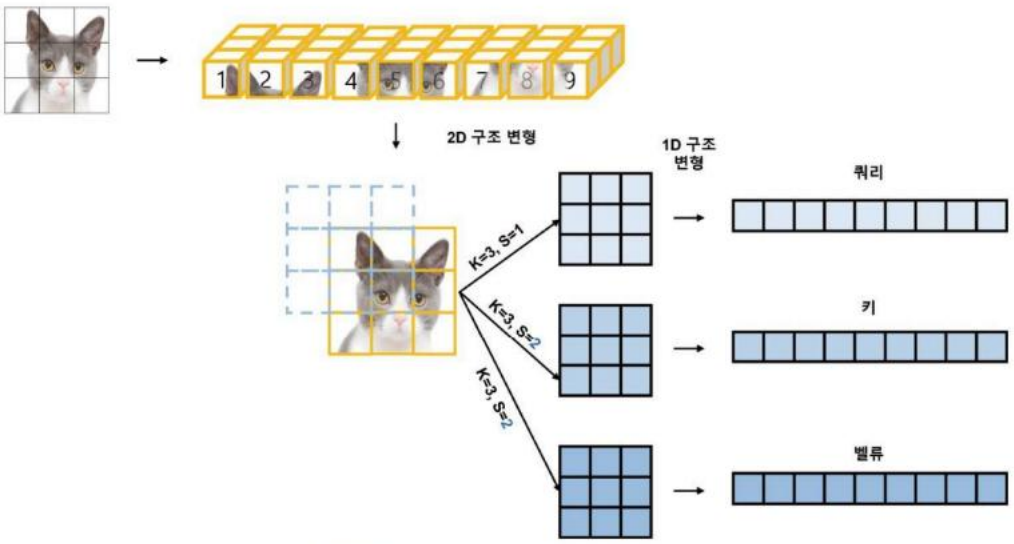
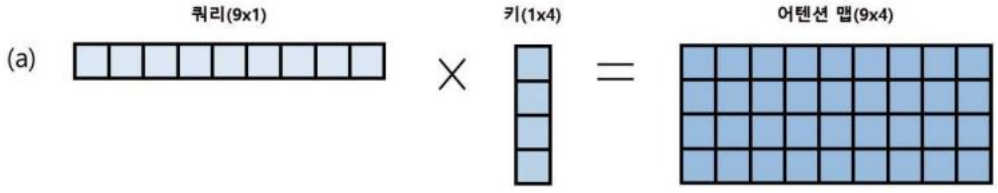



그림 10.16 2D 합성곱 기반의 축소된 어텐션 임베딩



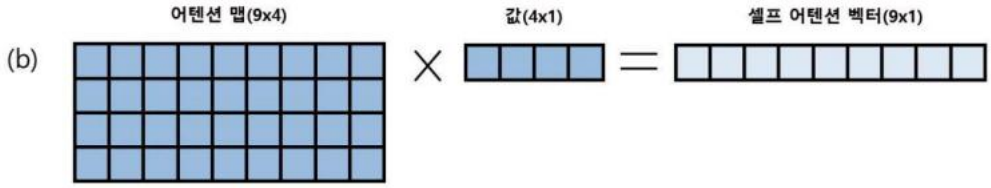




그림 10.17 축소된 셀프 어텐션 임베딩(열 × 행) 예시

## # CvT 실습 - 이미지 전처리

- 교재 사전 학습 모델 : microsoft/cvt-21

**AutoImageProcessor**

- CvT 사전 학습 모델의 전처리 정보 사용

**Resize**

- ViT 와 달리 size["shortest_edge"]를 이용해 짧은 변을 기준으로 이미지 크기를 변경

**Lambda**

- FashionMNIST 1 채널 이미지를 3 채널로 변경

**Normalize**

- 사전 학습 CvT 모델의 평균과 표준편차 사용

## # 사전 학습된 CvT 모델

**CvtForImageClassification**

- CvT 기반 이미지 분류 모델
- microsoft/cvt-21 사전 학습 모델을 불러와 FashionMNIST 10 개 클래스에 맞게 classifier 조정
- CvT 주요 구조 : cvt → encoder → stages → layernorm → classifier
- encoder 는 여러 개의 stage 로 구성

## # CvT 스테이지 구조

- 각 스테이지는 CvtEmbeddings 와 여러 CvtLayer 로 구성
- CvtEmbeddings : 합성곱 토큰 임베딩 + LayerNorm + Dropout
- CvtLayer : Attention + Intermediate + Output 구조
- 어텐션 내부에서 convolution_projection_query/key/value 를 사용
- 선형 Projection 과 합성곱 Projection 을 함께 사용하여 Q, K, V 를 생성

## # CvT 셀프 어텐션 출력 확인

- 입력 이미지 : [32, 3, 224, 224]
- 첫 번째 스테이지 패치 임베딩 출력 : [32, 64, 56, 56]
- 셀프 어텐션 입력은 [batch, token, channel] 형태로 변환
- 셀프 어텐션을 거친 뒤 각 위치의 특징을 반영한 출력 생성

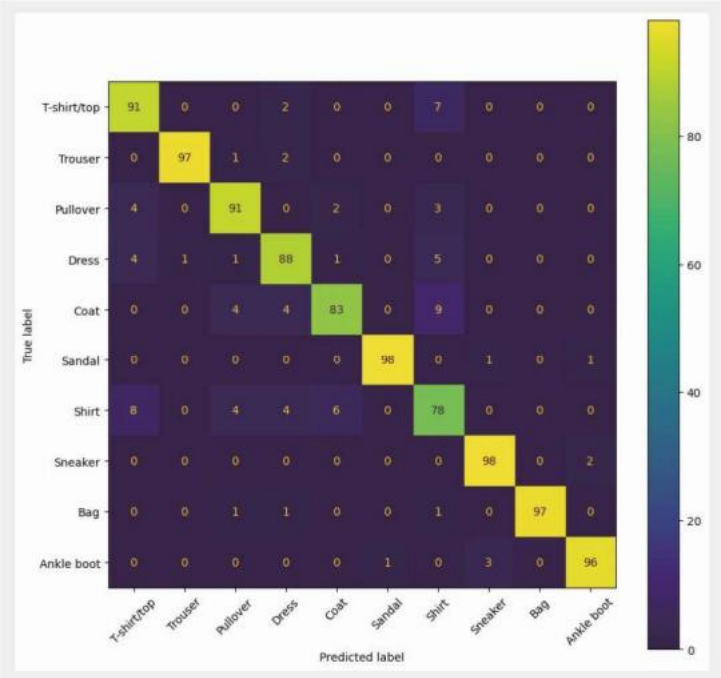
## # CvT 모델 학습

- ViT 와 동일한 Trainer 기반 학습 구조를 사용
- 모델 : microsoft/cvt-21
- 평가 지표 : Macro F1
- 3 epoch 동안 FashionMNIST 학습

| Epoch | Training Loss | Validation Loss | F1 |
|---|---|---|---|
| 1 | 0.7737 | 0.3269 | 0.8914 |
| 2 | 0.7008 | 0.2596 | 0.9172 |
| 3 | 0.6438 | 0.2460 | 0.9173 |

- F1 점수 : 0.9173

- F1 은 0.9159 에서 0.9173 으로 스윈 트랜스포머보다 소폭 개선




## # ViT / Swin Transformer / CvT 성능 비교

| 모델 | 모델 크기 | F1 | 추론 처리량 |
|---|---|---|---|
| ViT | 327.325 MB | 약 0.923 | 29.636 eval/s |
| Swin Transformer | 105.227 MB | 0.9159 | 58.048 eval/s |
| CvT | 120.791 MB | 0.9173 | 44.778 eval/s |

- FashionMNIST 실험에서 ViT 가 F1 은 가장 높지만 모델 크기가 가장 큼
- Swin Transformer 는 모델 크기가 작고 추론 처리량이 가장 높음
- CvT 는 F1 이 Swin 보다 소폭 높고, 모델 크기와 처리량은 ViT 와 Swin 의 중간 수준
- 교재는 각 모델의 구조적 차이와 함께 성능, 모델 크기, 처리 속도를 비교하여 어떤 모델이 항상 절대적으로 우수한 것이 아니라 문제와 환경에 따라 선택해야 함을 보여줌